

```

-- format: databaseName.tableName.columnName

-- Selects everything
select * from demo.pixar_films;

-- Selects particular column
select film, film_rating, box_office_worldwide from demo.pixar_films;

-- Where clause
select film, budget, box_office_worldwide from demo.pixar_films
where cinema_score = "A+";

-- And Operator
select ID, film from demo.pixar_films
where cinema_score = "A" and imdb_score >= 8.0;

-- Or Operator
select ID, film from demo.pixar_films
where cinema_score = "A" or imdb_score >= 8.0;

-- Not Operator
select ID, film from demo.pixar_films
where not cinema_score = "A" and not run_time <=90;

-- Like Operator
select ID, film, release_date from demo.pixar_films
where film like "%2%";

select ID, film, release_date from demo.pixar_films
where film like "Toy%";

select ID, film, release_date from demo.pixar_films
where film like "%3";

-- Order By
select film, budget, imdb_score from demo.pixar_films
order by film;

select film, budget, imdb_score from demo.pixar_films
order by film desc;

-- Limit Clause
select * from demo.pixar_films
order by budget
limit 10;

select * from demo.pixar_films
limit 2,5; -- 2 rows ke baad, 5 rows show karo

-- Between Operator
select film, release_date, run_time from demo.pixar_films
where run_time between 95 and 106;

-- In Operator
select film, cinema_score from demo.pixar_films

```

```

where cinema_score in ("A","A+");

select film, cinema_score from demo.pixar_films
where cinema_score in ("A","A+") and run_time >= 100;

-- Not In Operator
select film, cinema_score from demo.pixar_films
where cinema_score not in ("A","A+");

-- String Functions
-- concat (joining the string)
select concat(film, ", ", release_date) as film_details from demo.pixar_films;
select *, concat(film, ", ", release_date) as film_details from demo.pixar_films;

-- concat_ws (joining the string with delimiter)
select concat_ws(", ", film, release_date, imdb_score) as film_details from
demo.pixar_films;

-- length (string ke size count karne ke liye)
select film, box_office_worldwide, length(box_office_worldwide) as film_len from
demo.pixar_films;

-- upper (upper case me convert kar dega)
select film, upper(film) as film_upper from demo.pixar_films;

-- lower (lower case me convert kar dega)
select film, lower(film) as film_lower from demo.pixar_films;

-- left (left se kitne character)
select film, left(film,6) as film_left from demo.pixar_films;

-- right (right se kitne character)
select film, right(film,6) as film_right from demo.pixar_films;

-- mid (string ke beech se specific characters nikalne ke liye)
select film, mid(film,3,8) as film_mid from demo.pixar_films;

-- Data Aggregation
-- sum
select sum(box_office_worldwide) as total_films_revenue from demo.pixar_films;

-- count
select count(ID) as total_films from demo.pixar_films;

-- average
select avg(box_office_worldwide) as average_revenue from demo.pixar_films;

-- min and max
select min(box_office_worldwide) as min_revenue from demo.pixar_films;
select max(box_office_worldwide) as max_revenue from demo.pixar_films;

-- truncate (decimal number ko specified decimal places tak truncate karta hai, round
nahi karta)
select imdb_score, truncate(imdb_score,0) as imdb_truncate from demo.pixar_films;

```

```

-- ceil (number ko next highest integer tak le jaata hai)
select imdb_score, ceil(imdb_score) as imdb_ceil from demo.pixar_films;

-- floor (number ko nearest lower integer tak le jaata hai)
select imdb_score, floor(imdb_score) as imdb_floor from demo.pixar_films;

-- Date Functions
-- date
select date(release_date) as release_date from demo.pixar_films;

-- time
select time(release_date) as release_time from demo.pixar_films;

-- datediff (Difference b/w 2 dates)
select datediff(release_date, release_date) as date_difference from demo.pixar_films;

-- day
select release_date, day(release_date) as day from demo.pixar_films;

-- dayname
select release_date, dayname(release_date) as day_name from demo.pixar_films;

-- month
select release_date, month(release_date) as month from demo.pixar_films;

-- monthname
select release_date, monthname(release_date) as month_name from demo.pixar_films;

-- year
select release_date, year(release_date) as year from demo.pixar_films;

-- hour
select release_date, hour(release_date) as hour from demo.pixar_films;

-- minutes
select release_date, minute(release_date) as minute from demo.pixar_films;

-- second
select release_date, second(release_date) as second from demo.pixar_films;

-- CASE Operator
select film, box_office_worldwide, year(release_date) as release_year,
case
    when imdb_score >= 9 then "Masterpiece"
    when imdb_score >= 8 then "Excellent"
    when imdb_score >= 7 then "Very Good"
    when imdb_score >= 6 then "Good"
    else "Average"
end as "film_rating"
from demo.pixar_films;

-- GroupBy
select film_rating, count(ID) as total_films, sum(box_office_worldwide) as
total_revenue from demo.pixar_films
group by film_rating;

```

```

select film_rating, count(ID) as total_films, sum(box_office_worldwide) as
total_revenue from demo.pixar_films
group by film_rating
order by total_revenue desc;

-- Having Clause (Just like where clause, but for groupby)
select cinema_score, sum(box_office_worldwide) as total_revenue from demo.pixar_films
group by cinema_score
having total_revenue >= 10000000000;

-- Joins
-- First create table structure by right-clicking on Tables and then click on create
table.
-- Then add these records
INSERT INTO demo.pixar_films_cast VALUES
(1, 'John Lasseter'),
(2, 'John Lasseter'),
(3, 'John Lasseter'),
(4, 'Pete Docter'),
(5, 'Andrew Stanton'),
(6, 'Brad Bird'),
(7, 'John Lasseter'),
(8, 'Brad Bird'),
(9, 'Andrew Stanton'),
(10, 'Pete Docter'),
(11, 'Lee Unkrich'),
(12, 'John Lasseter'),
(13, 'Mark Andrews & Brenda Chapman'),
(14, 'Dan Scanlon'),
(15, 'Pete Docter'),
(16, 'Peter Sohn'),
(17, 'Andrew Stanton'),
(18, 'Brian Fee'),
(19, 'Lee Unkrich'),
(20, 'Brad Bird'),
(21, 'Josh Cooley'),
(22, 'Dan Scanlon'),
(23, 'Pete Docter'),
(24, 'Enrico Casarosa'),
(25, 'Domee Shi'),
(26, 'Angus MacLane'),
(27, 'Peter Sohn'),
(28, 'Kelsey Mann');

-- Inner Join
select pixar_films.film, pixar_films.box_office_worldwide, pixar_films_cast.director
from pixar_films inner join pixar_films_cast
on pixar_films.ID = pixar_films_cast.ID;

select pixar_films.cinema_score, count(pixar_films_cast.director) as director_count
from pixar_films inner join pixar_films_cast
on pixar_films.ID = pixar_films_cast.ID
group by pixar_films.cinema_score;

```

```

-- Left Join
select pixar_films.ID, pixar_films.film, pixar_films_cast.director from pixar_films
left join pixar_films_cast
on pixar_films.ID = pixar_films_cast.ID;

-- Right Join
select pixar_films_cast.ID, pixar_films.film, pixar_films_cast.director from
pixar_films right join pixar_films_cast
on pixar_films.ID = pixar_films_cast.ID;

-- Cross Join
select * from pixar_films cross join pixar_films_cast;

-- Set Operators
-- UNION
select ID from pixar_films
union
select ID from pixar_films_cast;

-- UNION ALL
select ID from pixar_films
union all
select ID from pixar_films_cast;

-- INTERSECT
select ID from pixar_films
intersect
select ID from pixar_films_cast;

-- EXCEPT
select ID from pixar_films
except
select ID from pixar_films_cast;

-- SubQueries in MySQL
select * from demo.pixar_films
where budget > (select avg(budget) from demo.pixar_films);

select ID, film from demo.pixar_films
where ID in (select ID from demo.pixar_films_cast);

-- Views in MySQL
create view cinema_budget as
select cinema_score, sum(budget) as total_budget from demo.pixar_films
group by cinema_score;

select * from cinema_budget;

-- Dropping View
drop view cinema_budget;

-- Stored Procedure
-- Creating
delimiter &&
create procedure getFilms()

```

```

begin
    select * from demo.pixar_films;
end &&
delimiter ;

-- Calling
call getFilms()

-- in (input)
delimiter &&
create procedure get_limit(in var int)
    select * from demo.pixar_films limit var;
end &&
delimiter ;

call get_limit(5);

-- out (output)
delimiter &&
create procedure get_budget(out var bigint)
begin
    select sum(budget) into var from demo.pixar_films;
end &&
delimiter ;

call get_budget(@m);

select @m;

-- inout (input-output)
delimiter &&
create procedure get_film_name(inout var int)
begin
    select * from demo.pixar_films where ID = var;
end &&
delimiter ;

set @m = 4;
call get_film_name(@m);
select @m;

-- Dropping a Procedure
drop procedure if exists get_film_name;

-- Windows Functions
-- Without Window Functions
select cinema_score, film_rating, sum(box_office_worldwide) from demo.pixar_films
group by cinema_score, film_rating;

-- With Window Functions
select cinema_score, film_rating, sum(box_office_worldwide)
over (partition by cinema_score order by film_rating) from demo.pixar_films;

-- ROW_NUMBER() - Har row ko unique number deta hai
select cinema_score, film_rating, row_number()

```

```
over (partition by cinema_score order by film_rating) from demo.pixar_films;

-- RANK() - Same values ko same rank deta hai, beech mein gap aa sakta hai.
select cinema_score, film_rating, rank()
over (partition by cinema_score order by film_rating) from demo.pixar_films;

-- DENSE_RANK() - Same values ko same rank deta hai, but gap nahi aata.
select cinema_score, film_rating, dense_rank()
over (partition by cinema_score order by film_rating) from demo.pixar_films;
```